MISSION CRITICAL EMAIL — ZIMBRA MIGRATION SOFTWARE

Static Code Review & Control-Flow Analysis

Mission Critical Email — Zimbra Migration Software

Software reviewed	Mission Critical Email Backup-Restore — the migration binary (Go, release line 15.x). A single statically-linked executable; no runtime dependencies, no bundled libraries.
Size	~9,600 lines of Go (product code) + ~7,100 lines of automated tests, across 12 internal packages.
Prepared by	Mission Critical Email LLC — Engineering
Date	1 November 2026 (supersedes the 24 July 2026 edition)
Classification	May be shared with prospective and active customers, and with their security and audit teams.

This report summarizes a structured review of the migration software's architecture, control flow, error handling, and conformance to Go engineering and operational best practices. It is provided so that organizations evaluating the software can have confidence that it has undergone disciplined engineering review appropriate to production-grade migration tooling.

Lineage note. The software was originally delivered as a field-proven bash orchestration script. It has since been re-implemented as a compiled Go binary, which is now the production line. The Go binary was first **certified observationally equivalent** to the frozen bash reference (Section 7.2); since that certification, the Go line has moved deliberately *beyond* the reference — both with new capabilities and by finding and fixing latent defects the reference itself carried for years (Section 7.4). Every divergence from the reference is individually documented, rationalized, and pinned by regression tests in the engineering record.

What is new in this edition. Since July 2026 the code base has been through a second review cycle: a third static analyzer (`staticcheck`) was added to the toolchain, **native fuzz testing** was introduced for every parser of external data (13 targets, ~10 million executions, no crashes), and the restore side was restructured so that every file of the backup format is read through one typed reader package. The cycle produced four findings; three are fixed and one is scheduled, and all four are listed in Section 7.5 so that a reviewer can see exactly what was found and what was done about it.

Results at a glance

Area	Result
Control-flow analysis	Pass

Area	Result
Error handling & recovery	Pass
Static analysis (`go vet`)	Pass (0 findings)
Static analysis (`staticcheck` 2025.1)	Pass (2 findings in October 2026; both fixed; now 0)
Formatting conformance (`gofmt`)	Pass (clean)
Automated test suite	Pass (211 tests, 12 packages)
Fuzz testing (13 native fuzz targets)	Pass (no crash, panic, hang, or memory exhaustion)
Equivalence to field-proven oracle	Pass (certified at parity)
Boundary-assumptions audit	Complete (Section 3, h)
Independent multi-agent review	Pass (2 findings; fixed)
Data-safety properties	Pass
Security posture (code-level)	No findings
Software composition (third-party code)	None — Go standard library only
Critical defects	0
Open findings from the October 2026 cycle	1, non-critical, scheduled (Section 7.5, SA3)

Scope of this result: the reviewed binary's source, as written. See Section 2 (Scope & Limitations) for what this review does and does not cover.

---

1. Purpose & scope

The migration software performs disaster-recovery-style backup and restore of a Zimbra environment — Classes of Service, domains, accounts, aliases, signatures, distribution lists, per-folder data (calendars, contacts, tasks, briefcase), share grants and mountpoints, DKIM keys, per-user two-factor authentication state, out-of-office auto-reply settings, RSS/ICS folder subscriptions, and (via IMAPSYNC) mailbox message content. A license may scope a binary to an exact set of mail domains for staged, multi-wave migrations of multi-tenant systems; a scoped binary exports and restores *only* those domains and refuses out-of-scope work on both the backup and restore sides. See the Operator Guide for more detail.

This review covers the backup/restore migration binary. Its goals:

- Map the control flow end to end (entry points, dispatch, exit paths).
- Enumerate the error-handling and recovery paths.
- Run the standard Go static-analysis, fuzzing, and test tooling and report the results.
- Assess conformance to defensive-coding and data-safety best practices.

- Confirm behavioral equivalence to the field-proven reference implementation.

2. Scope & limitations

This review evaluated the migration binary's Go source (~9,600 lines of product code across 12 internal packages, plus ~7,100 lines of automated tests). It comprised static analysis, control-flow analysis, error-path analysis, automated testing, fuzz testing, equivalence verification against the field-proven reference, and engineering best-practice assessment of that source.

The review did **not** include, and no assurance is expressed regarding:

- Penetration testing or runtime exploitation of a deployed system.
- Security, hardening, or configuration of the Zimbra servers, the operating systems, or the network they run on.
- The external programs the binary invokes (Zimbra CLI / `zmprov` / `zmmailbox`, `curl`, `ldapsearch` / `ldapmodify`, GNU Parallel) — these are provided and patched by the host OS and Zimbra installation.
- IMAPSYNC, the separate third-party utility used for mailbox-message transfer, and the four open-source preparation scripts published alongside this document (broken-shares cleanup, IMAP preflight, CoS-override report, seat count), which are reviewed separately.
- Customer-specific environments, configurations, or customizations.

Findings therefore apply to the reviewed binary as written and should not be read as an assessment of the surrounding platform, third-party tooling, or any particular deployment.

3. Methodology

The review applied the techniques standard to evaluating production Go software:

- **(a) Static analysis with `go vet`** — the Go toolchain's built-in correctness analyzer (suspicious constructs, printf mismatches, lock misuse, etc.) — across all packages.
- **(b) Static analysis with `staticcheck`** (release 2025.1) — the de-facto standard third-party analyzer for Go, covering a much larger rule set than `go vet`: nil-dereference paths, unused code, deprecated APIs, and several hundred correctness and style checks. Added to the toolchain in October 2026.
- **(c) Formatting conformance with `gofmt`** — the canonical Go formatter — so the entire codebase is mechanically consistent and diff-clean.
- **(d) Automated test suite** — 211 test functions across the 12 internal packages, including table-driven tests carried over verbatim from the reference implementation's unit-test fixtures (the parser and inversion logic most prone to subtle bugs) and fixture-driven tests of the typed backup-format reader, run with the standard `go test`.

- **(e) Fuzz testing** — 13 native Go fuzz targets (`go test -fuzz`) over every parser of external data: Zimbra folder listings, mountpoint targets, share grants, account attribute dumps, LDIF, distribution lists, DKIM records, signatures, share-inventory inversion, the SOAP auth-token extractor, the IMAP modified-UTF-7 encoder, and the IMAPSYNC exclude-pattern generator. Each target asserts the parser's invariants (no panic; round-trips; generated regexes compile and match exactly what they should). The seed corpora are committed with the source and the targets run in the normal test suite.
- **(f) Equivalence certification** — a record/replay harness compares the external-command transcript and generated-file tree produced by this binary against the frozen, field-proven bash reference for the same input (Section 7.2).
- **(g) Full control-flow mapping** — a complete package/function map and call graph, with explicit identification of entry points, normal exit paths, and error/abort paths.
- **(h) Boundary-assumptions audit** — three independent structured surveys of every external interface: each command the binary invokes, each parser of external output, each constructed payload, and the provenance of every test fixture. Every wire-format assumption was cataloged in a register and classified (pinned to a real capture / synthetic fixture / unverified); each finding was fixed or explicitly documented as an accepted, bounded risk. The register ships with the source as a living engineering document.
- **(i) Seam inventory** — a file-and-line inventory of every place the restore side reads the backup directory, every place backup and restore duplicated a helper, and every point at which the binary touches the outside world. Conducted in October 2026 ahead of a planned extension of the product; its findings and the resulting restructuring are in Section 7.5.
- **(j) Independent multi-agent adversarial review** — the full change set was additionally submitted to an independent, automated multi-agent code review (multiple analysis agents with no access to the authors' context or assumptions). It produced two verified findings; both were accepted, fixed, and pinned with regression tests. Notably, it re-flagged none of the documented accepted risks and contradicted none of the register's "verified safe" determinations.
- **(k) Best-practice conformance** — defensive coding, fail-fast pre-flight validation, idempotency, least privilege, and auditability.

4. Architecture overview

The software is driven by two actions and four modes from a single entry point:

```
Action : backup (-b)  |  restore (-r)
Mode   : pre | post | full | shares
```

- **pre** — provisioning (Classes of Service, DKIM, domains, accounts, aliases, distribution lists): the half of a migration done while users remain on the source system.

- **post** — user data (per-folder calendars/contacts/tasks/briefcase, filters, signatures, share grants, mountpoints): the half done at or after cutover.
- **full** — pre + post.
- **shares** — standalone re-run of grants + mountpoints.

This pre/post separation is deliberate: it supports a staged, near-zero-downtime production cutover (bulk-copy while users stay on the source, then a fast delta at cutover), rather than requiring a single long outage.

The implementation is organized into 12 focused internal packages:

Package	Responsibility
`cli`	Argument parsing and validation.
`operator`	Interactive confirmations and credential collection (TTY prompts, or environment variables when unattended).
`license`	Embedded, signature-verified license: host binding, validity window, seat cap, domain scope.
`zimbra`	The single external-command choke point: every `zmprov` / `zmmailbox` / `curl` / LDAP invocation, UTF-8 locale enforcement, argument quoting, optional transcript recording, SOAP authentication and the proxy data path.
`backup`	The export side: enumeration, attribute dumps, per-folder data, shares, DKIM, two-factor state, and generation of the IMAPSYNC bundle.
`exportfmt`	The typed reader of the backup directory — one accessor per file of the export format, so that every consumer reads each file through exactly one piece of code (new, October 2026).
`restore`	The import side: provisioning, per-folder data import, grants and mountpoints, DKIM, advanced attributes, deferred quota and status re-arming.
`parse`	The parsers of Zimbra tool output, pinned by table-driven tests and fuzz targets.
`text`	Shared line/field helpers used by backup and restore (new, October 2026).
`parallel`	Bounded fan-out with per-job timeouts.
`logging`	Serialized logging to a dated main log and a separate error log.
`guard`	Host pre-flight checks: UTF-8 locale, required commands, free disk (see Section 7.5, SA3).

All external commands route through the single choke-point client in `zimbra`, which is where argument quoting and the UTF-8 locale are enforced once and where an optional, complete command transcript is captured (Section 8).

5. Control flow & call graph

5.1 Execution path (entry to exit)

```
START
  |
  |-- Pre-flight gates (fail fast, before any data operation):
  |     1. Argument parsing & validation  (rejects unknown modes/arguments)
  |     2. Embedded license verification  (signature, validity window, and
  |        binding to THIS host for the requested action; aborts otherwise)
  |
  |-- Logging: dated main log + dedicated error log, in the job directory
  |
  |-- External-command client: the single os/exec choke point, with the
  |     UTF-8 locale forced on every child process and, under --record,
  |     a transcript recorder attached
  |
  |-- Operator confirmations (backup pre/full): broken-share cleanup done;
  |     IMAP preflight applied -- declined = clean abort, nothing touched
  |
  |-- Credential + session establishment (restore/post/shares, backup post/full):
  |     - Administrator authentication (SOAP), proxy session setup
  |     - Interactive (TTY) prompts OR environment-variable values for
  |       non-interactive (cron/nohup) execution -- detected and handled
  |
  |-- Account reachability validation:
  |     - Probes every candidate account; unreachable/orphaned entries are
  |       quarantined to a skip list rather than failing the run
  |
  |-- ACTION x MODE dispatch -> phased execution (see 5.2)
  |
  |-- Normal completion -> success log -> exit 0
END
```

A note on the host pre-flight checks (UTF-8 locale present on the host, required commands on `PATH`, free disk): these exist as the `guard` package but, as of this edition, are **not yet invoked** from the entry point — see finding SA3 in Section 7.5. The locale is nevertheless enforced for every child process at the command boundary, and a missing command surfaces as an immediate, logged failure of the first operation that needs it.

5.2 Functional subsystems (by package)

Subsystem	Packages
Startup & utilities	`cli`, `license`, `guard`, `logging`
Authentication & transport	`zimbra` — administrator SOAP authentication, per-user delegated tokens, curl-via-proxy streaming for folder export/import (the choke-point client)
Export (backup side)	`backup` — COS, domain→COS mapping, account attributes, per-folder data, share grants, mountpoints, DKIM; per-user backup orchestration; IMAPSYNC bundle generation

Subsystem	Packages
Import (restore side)	`restore` over `exportfmt` — COS/domain/account/alias provisioning; per-folder data import; grants/mountpoints; DKIM; advanced attributes; distribution-list members; proxy-timeout preflight + import-failure tally
Parsers	`parse`, `text` — the output parsers for Zimbra tool output, pinned by table-driven tests and fuzz targets
Parallelism	`parallel`

5.3 Parallelism

Per-account work (export, folder restore, grants, mountpoints) is optionally parallelized. Each parallel operation carries a per-job timeout backstop so that no single stalled operation can block an entire phase indefinitely. A sequential path (`-j 1`) is always available.

6. Error-handling & recovery analysis

The software treats failure handling as a first-class design concern.

6.1 Centralized external-command boundary. Every external command (Zimbra CLI, curl, LDAP tools) is issued through a single choke-point client that returns a typed result (output + exit status). A non-zero exit is a value the caller must handle — never a silent success — and the same boundary enforces the UTF-8 locale and argument quoting once, eliminating an entire class of shell-quoting and encoding bugs by construction.

6.2 Detection coverage. Every external-tool invocation is checked and its failure is logged. Errors and warnings are written to **both** a dated main log and a dedicated error log for fast post-run review.

6.3 Retry with backoff. Transient network operations (folder import via proxy) retry up to a configured maximum with escalating backoff, distinguishing a momentary service blip from a hard failure.

6.4 Tolerated, deliberate partial failures. Selected operations intentionally tolerate and log a non-fatal failure rather than aborting the whole migration — e.g., one account's optional data not blocking the other thousands. Each is explicit and logged, never silent.

6.5 Idempotency & resumability. The restore is idempotent for mailbox content: folder import uses a skip-existing strategy, so a run can be safely repeated or resumed after an interruption without duplicating data. The standalone "shares" mode exists precisely to re-run the grant/mountpoint phase independently. (Class-of-Service restore is non-destructive rather than strictly idempotent: an existing destination COS is never overwritten — a name collision is created as `<name>_restored` — so repeating a full run leaves only harmless, deletable COS duplicates, never altered data.)

6.6 Fail-fast pre-flight. The license (including an optional per-customer mailbox-count cap), the operator's confirmation that the source has been prepared, administrator authentication, and account reachability are all validated **before** any data operation, so misconfiguration is caught up

front rather than mid-migration. Restore additionally surfaces a proxy upstream-timeout advisory before large imports, and an end-of-run summary of any folder-import failures.

6.7 One reader for the backup format. As of October 2026 every file the restore consumes is read through the `exportfmt` package — one accessor per file, each with a fixture-driven test, and a smoke test that reads a real backup end to end. A format change is therefore made in exactly one place, and a malformed or truncated file is handled identically wherever it is read. The restructuring was proven behaviour-preserving by the equivalence harness (identical command transcript, identical generated files).

7. Static analysis, testing & equivalence results

7.1 Toolchain results

Check	Result
Static analysis (`go vet`)	0 findings across all packages.
Static analysis (`staticcheck` 2025.1)	0 findings. (Its first run, in October 2026, reported two — Section 7.5.)
Formatting (`gofmt`)	Clean — the entire codebase is canonically formatted.
Compilation	Builds clean as a single statically-linked binary (CGO disabled); one binary serves both Debian/Ubuntu and RHEL/Rocky targets.
Automated tests (`go test`)	211 test functions across all 12 internal packages **pass**.
Fuzz testing (`go test -fuzz`)	13 targets, ~10 million executions in the October 2026 campaign: **no crash, panic, hang, or memory exhaustion**. The targets stay in the suite and run on every build with their committed seed corpora.
Test-to-code ratio	~7,100 lines of tests to ~9,600 lines of product code.

Test coverage emphasizes the highest-risk logic — the Zimbra-output parsers and the share-inversion logic — via table-driven tests carried over from the reference implementation's unit-test fixtures, extended with fixtures drawn from **real** protocol captures taken during production migrations (a policy adopted after the boundary audit: parsers of external formats are pinned to actual wire data, not merely plausible synthetic data), and since October 2026 by the fuzz targets, which exercise every parser with millions of mutated and hostile inputs.

Software composition / dependencies. The binary is built from the Go standard library **only** — no third-party modules are compiled in — so there is no software-composition (SCA) dependency tree to scan and no transitive-dependency supply-chain surface. The one remaining dependency is the Go toolchain itself, which is pinned per release and is what the Go vulnerability database check (`govulncheck`) covers for a standard-library-only module. The external programs the binary invokes

(Zimbra CLI, curl, LDAP tools, GNU Parallel) are provided and patched by the host OS and Zimbra installation, and fall under the platform boundary stated in Section 2.

No defects of correctness, data-safety, or security severity were identified.

7.2 Equivalence to the field-proven reference

The migration logic was first delivered and field-hardened as a bash orchestration script. The Go binary is held to a strict **equivalence** standard: for a given input, a record/replay harness verifies that the binary issues the same ordered set of external commands and produces the same generated files as the frozen bash reference. Result: an identical command sequence and byte-identical generated output, apart from a small, **documented** set of deliberate improvements — each recorded, rationalized, and test-pinned in the engineering record. This gave the Go binary an unusual assurance basis at its baseline: it was measured against an implementation already proven against real customer data. Since certification, the line has deliberately diverged from the reference where the reference was found to be *wrong* (Section 7.4); equivalence remains the documented baseline, not a ceiling. The same harness is what proved the October 2026 restructuring (Section 6.7) behaviour-preserving.

7.3 Field validation

Beyond automated testing, the binary is exercised end to end against real Zimbra environments. As of this edition that includes: multiple live customer production cutovers completed on this line; repeated clean-slate migration cycles of a multi-hundred-account production Network Edition system to a FOSS destination, with source/destination trees compared folder by folder and attribute by attribute; and a staged-wave cycle using a domain-scoped binary, verifying both the scoped export (only licensed domains and their data travel — including that other tenants' DKIM signing keys do **not**) and the restore-side refusal of out-of-scope backups. Validation fixtures deliberately include an 18-year-old power-user dataset (non-ASCII folder names, alias-identity shares, legacy folder metadata, feed subscriptions) chosen precisely because it surfaces edge cases that clean test data cannot. Field validation is an ongoing part of Mission Critical Email's engineering process, and its results are recorded per release in the engineering record.

Locale and character-set handling is explicit throughout: the binary forces a UTF-8 locale on every external command it runs, protecting against multi-byte data corruption at the boundary where it would occur. This extends to protocol-level encodings: IMAP wire folder names (RFC 3501 modified UTF-7) and LDAP base64-encoded values are handled explicitly, with the encoders/decoders pinned to real captured protocol data and fuzzed for round-trip correctness.

7.4 Defects inherited from the reference: found and fixed

A distinctive property of this line's engineering record: the audit and field-validation program identified a number of **latent** defects that the field-proven bash reference itself had carried silently for years — including several silent-data-omission paths (data classes that never migrated, with no error reported). Each was fixed in the Go line, recorded as a documented divergence from the reference, and pinned with a regression test built from the real-world data that exposed it. The

engineering posture is explicit: fidelity to the reference was a porting instrument, not a goal — once parity was certified, inherited defects are fixed as a matter of course and the divergence ledger is the record of proof.

7.5 The October 2026 review cycle: findings and dispositions

Every finding of the cycle is listed, including the one that is not yet closed. None is a correctness, data-safety, or security defect.

#	Source	Finding	Disposition
SA1	`staticcheck`	Possible nil dereference on an error path in the **license-generation build tool** (not in the shipped binary) — a false positive in practice, because the preceding call exits the process, but the analyzer could not know that.	**Fixed**: the exit is now explicit to the analyzer.
SA2	`staticcheck`	An unused local type left behind by an earlier shape of the COS batch builder.	**Fixed**: dead code removed.
SA3	Seam inventory	The host pre-flight checks (UTF-8 locale present on the host, required commands on `PATH`, free disk) are implemented in the `guard` package but never invoked from the entry point. The July 2026 edition of this document listed them among the startup gates; that was inaccurate and is corrected in this edition. Impact is bounded: the locale is forced on every child process regardless, so tool output stays parseable, and a missing command fails loudly at first use. On a host whose locale packages are absent, however, the Zimbra tools can still mangle non-ASCII data — which is exactly what the host assertion exists to prevent.	**Open, scheduled**: the three checks are wired into the entry point in the next release (15.11), with a test. Until then the Operator Guide's prerequisite — run as the `zimbra` user on a host with the `en_US.UTF-8` locale installed — stands in for the automated check.
FZ1–FZ3	Fuzz testing	Three cases where a parser *accepts* input that the real Zimbra tools never emit (an empty mountpoint name; a one-field line after a `zmmailbox` prompt treated as a grant; a `>` admitted inside a SOAP auth token). None causes a crash, a wrong result on real input, or a data-safety issue; each is a tolerance that could, with a future Zimbra version that adds new console chatter, let an unexpected line be mis-read.	**Documented**: each is pinned in its fuzz target as the current contract and recorded in the divergence ledger; the tightenings are queued as deliberate divergences with fixtures, per the equivalence doctrine (Go behaviour changes only when recorded).

8. Safety & data-protection properties

- **Non-destructive posture.** Backup is **read-only** to the source Zimbra itself — it never modifies source mail, LDAP, or configuration; its only writes are the backup output, into a directory of the operator's choice on local disk. Restore provisions and adds to the destination. Destructive

mailbox operations are confined to the IMAPSYNC tooling and are gated by explicit, phase-specific flags.

- **Idempotent & resumable** (Section 6.5).
- **Dry-run capable.** A full provisioning + structure migration can be exercised **without** moving mailbox content, so a customer can validate the outcome before committing to a cutover.
- **Least privilege.** Runs as the unprivileged Zimbra service user, needs no root, installs nothing into Zimbra, and is license-gated to authorized hosts. It uses dedicated migration administrator credentials — a backup collects two sets, one for the IMAPSYNC mail transfer and one for the proxy data path. These may be the same account only if that account has two-factor authentication disabled (the proxy data path cannot authenticate a 2FA-enabled account; a *separate* IMAPSYNC account may keep 2FA by using an app-specific password). The binary's only network egress is to the Zimbra reverse proxy of the systems being migrated — no telemetry, no external calls.
- **Fully auditable.** Comprehensive, timestamped, serialized logging with a separate error log; every created and skipped item is recorded. In addition, the binary can emit a complete machine-readable transcript of **every** external command it executes (enabled with the `--record` flag). The mechanism is structural: every external command the binary runs already flows through its single command choke point, and with recording enabled that choke point appends one JSON line — the program, its full argument list, and any data fed on standard input — to the transcript file before executing the command. Completeness is therefore by construction: there is no second code path by which a command could run unrecorded. A reviewer can thus confirm exactly what the binary does on their systems. (Such transcripts capture command arguments verbatim, including credentials, and are documented to be handled as secrets.)
- **Self-identifying.** Every delivered binary prints its own embedded, signature-verified license — domain scope, authorized hosts, seat cap, validity window — on request, on any machine. Delivery integrity is verified by a SHA-256 checksum command generated at build time for the operator to run verbatim.

9. Summary & conclusion

The migration software exhibits the characteristics expected of production-grade operational tooling: a single well-defined entry point with fail-fast license and argument validation; a clear action/mode dispatch; a centralized external-command boundary that eliminates shell-quoting and encoding defects by construction; comprehensive error handling with retry-with-backoff for transient conditions; idempotent, resumable, non-destructive operation; and, since October 2026, a single typed reader for its own backup format. Two static analyzers report zero findings, the codebase is canonically formatted, an automated suite of 211 tests passes, and 13 fuzz targets have driven roughly ten million mutated inputs through every parser of external data without a single crash. The binary was certified observationally equivalent to the field-proven reference implementation at its baseline, has since been hardened by a structured boundary-assumptions

audit and an independent multi-agent adversarial review (two findings, both fixed), and has repaired latent defects the reference itself carried — each divergence documented and regression-pinned.

Within the scope stated in Section 2, this review identified no critical defects. One non-critical finding from the October 2026 cycle (SA3) remains open and scheduled, and is disclosed above so that it can be verified closed in the next edition.

The software is additionally subjected to ongoing field validation against real Zimbra environments as part of Mission Critical Email's engineering process and actual, ongoing customer migrations.

Mission Critical Email LLC. This document describes software that is the proprietary, copyrighted intellectual property of Mission Critical Email LLC.